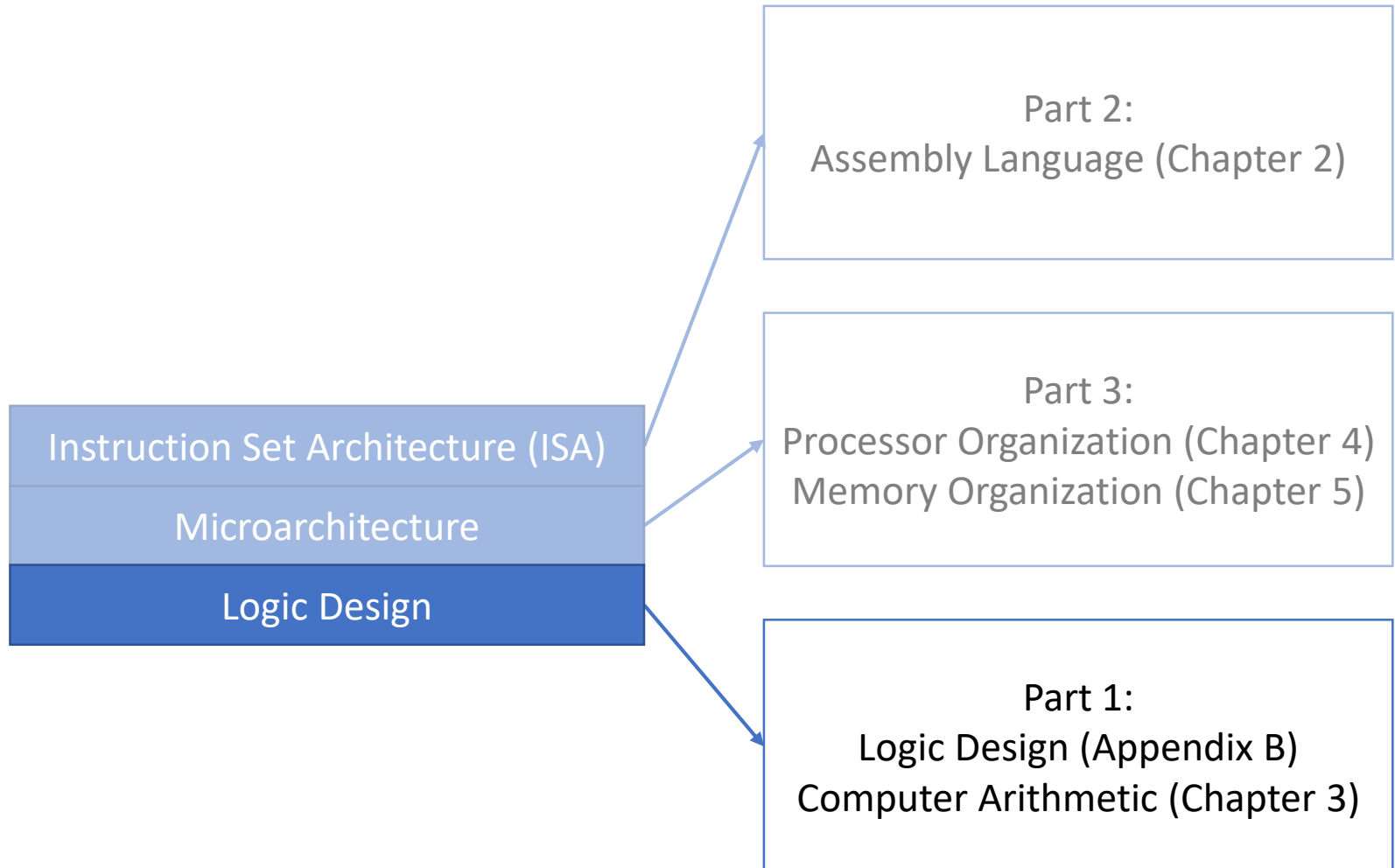


Lecture 13: Floating Point Representations

CMPS 221 – Computer Organization and Design

Last Time: Multiplication



Recall: Multiplication

Multiplicand:		1101
Multiplier:	×	1011

		1101
		1101
		0000
		1101

Product:		10001111

How to implement multiplication in hardware?

Implementing Multiplication

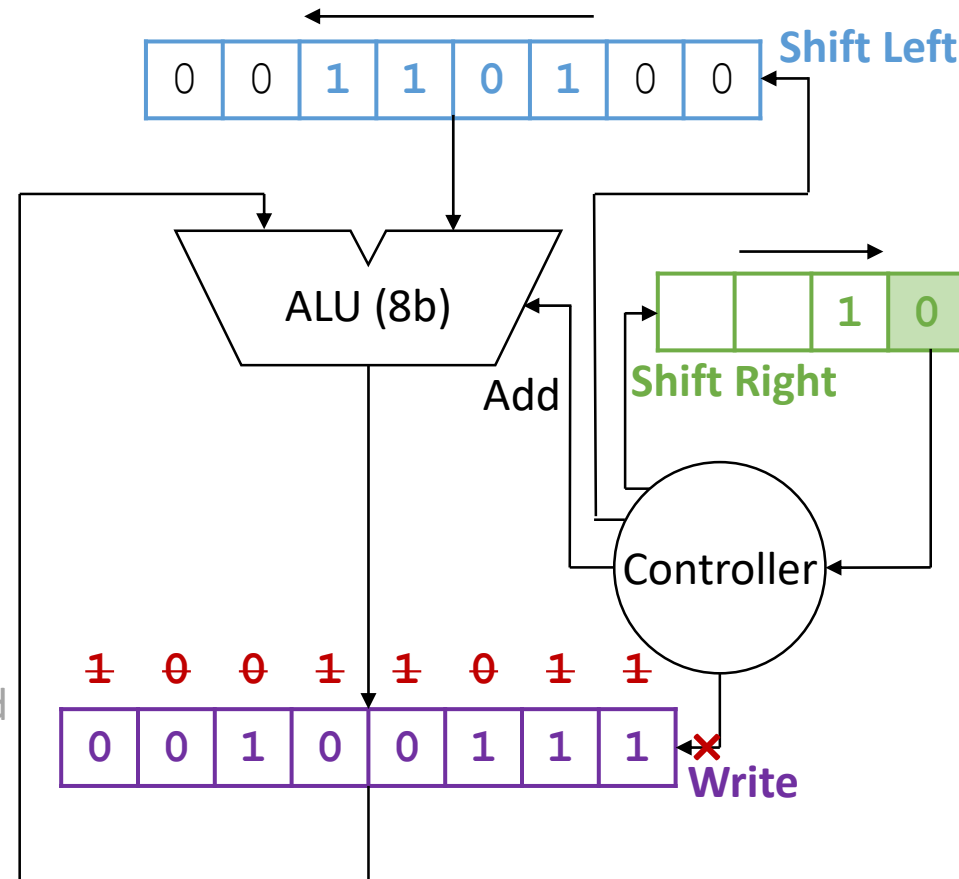
Multiplicand: 1101
Multiplier: × 1011

Cycle 0: 00001101 → + 00000000
 Cycle 1: 00011010 → + 00001101
 Cycle 2: 00110100 → + 00100111
 01101000 → + 00100111

Product: 10001111

Overall approach:

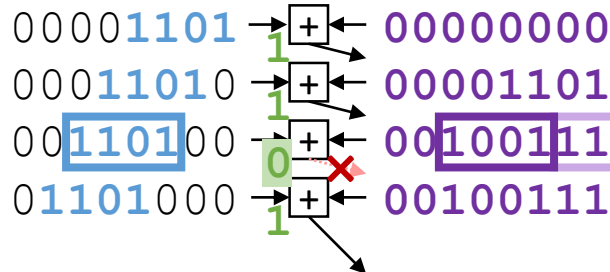
- Shift **multiplicand** left on every cycle and add it to **product**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum



Multiplication with Less Hardware

Multiplicand: 1101
Multiplier: × 1011

Cycle 0:

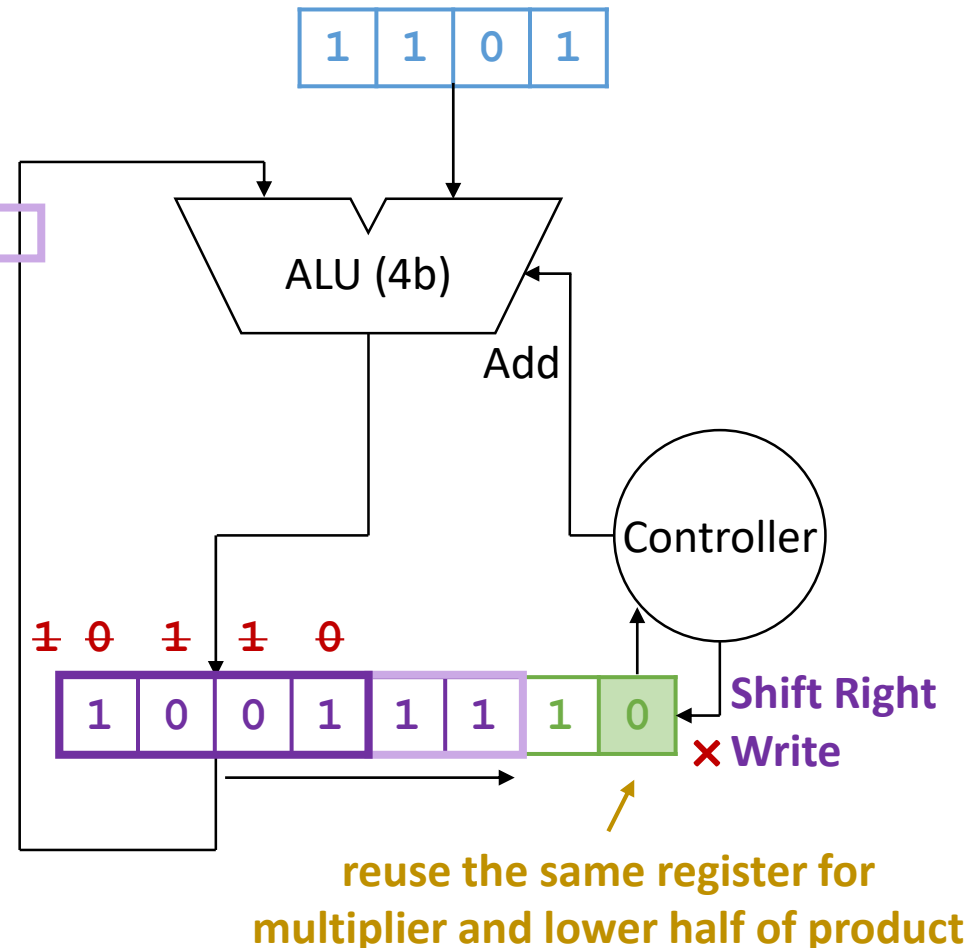


Product:

10001111

Overall approach:

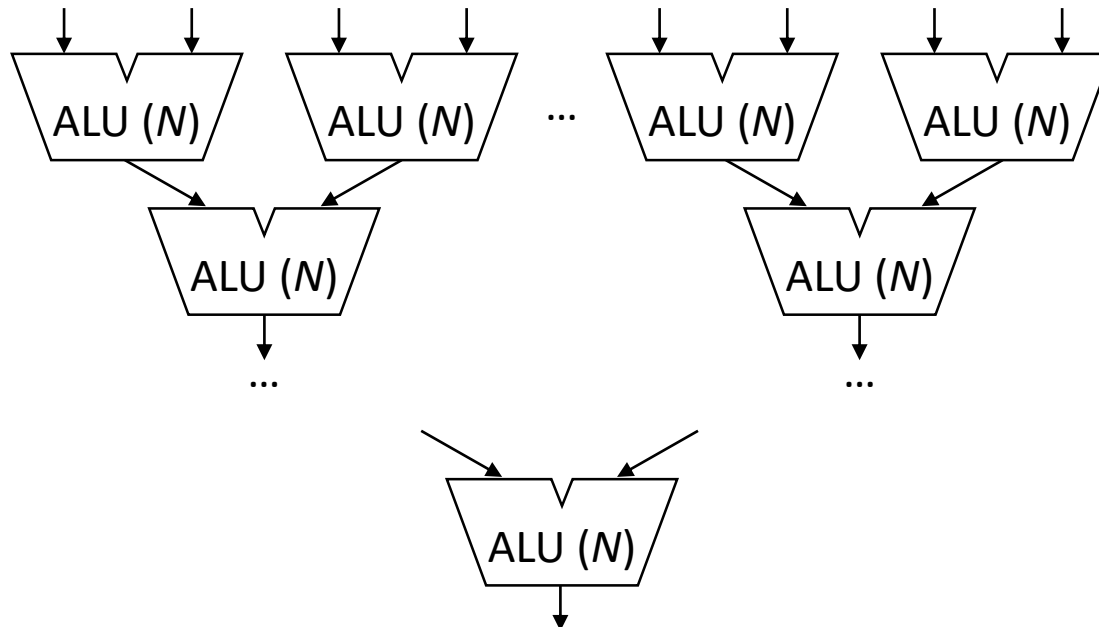
- Shift product window left (shift **product** right) on every cycle and add it stationary **multiplicand**
- Use **multiplier** bit to choose whether or not to update the **product** with the sum



Multiplication Faster

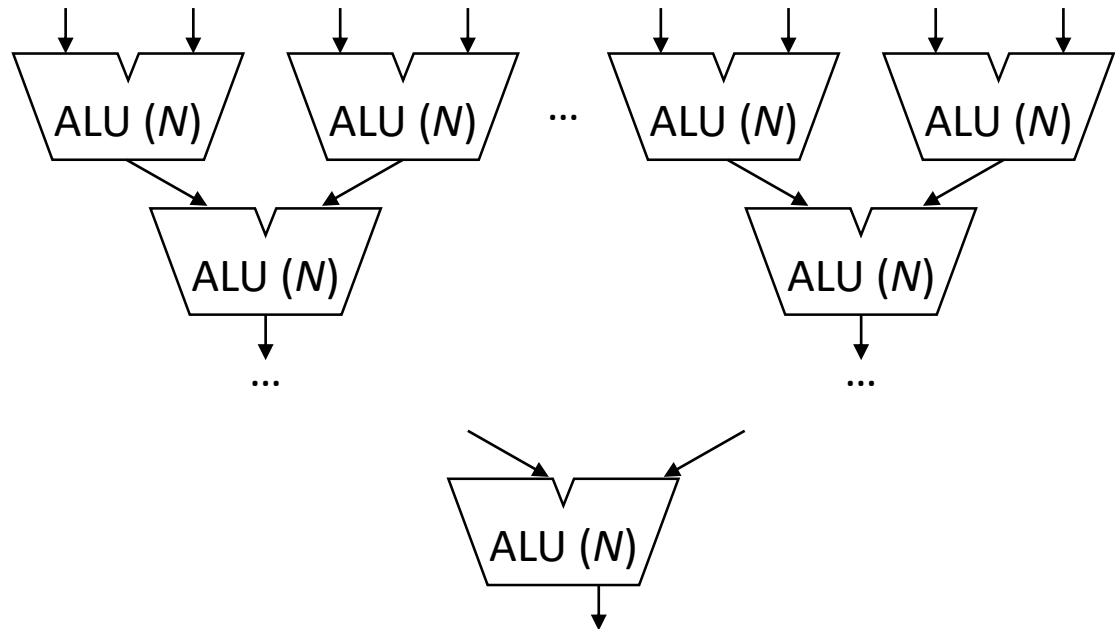
Multiplicand: 1101
Multiplier: × 1011

00001101
00011010 → + → 00100111
00000000
01101000 → + → 01101000 → + → **Product: 10001111**

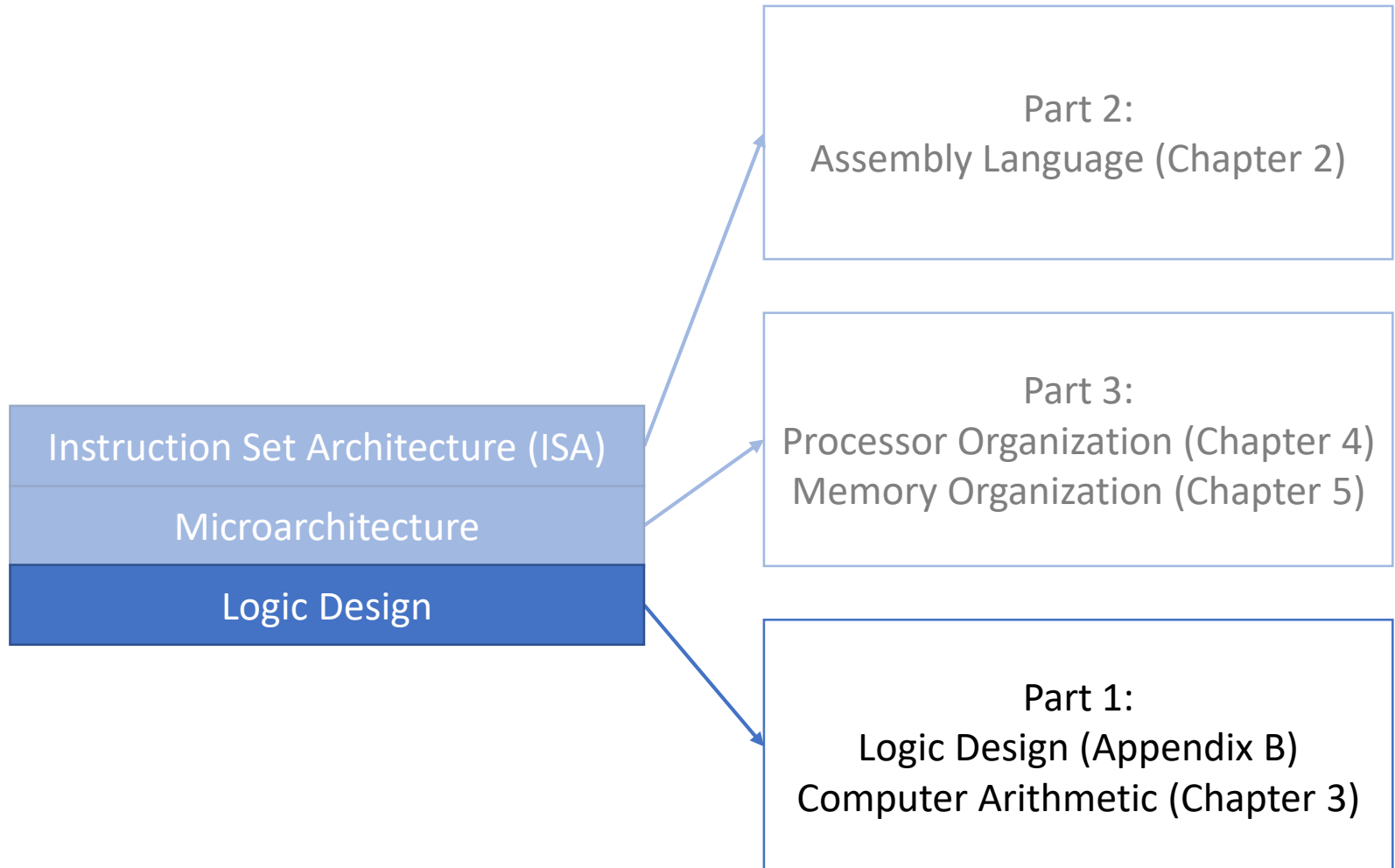


Cost of N -bit Multiplication

- First design:
 - ALU: $2N$
 - Registers: $5N$
 - Time: N
- Second design:
 - ALU: N
 - Registers: $3N$
 - Time: N
- Third design:
 - ALU: $N \times (N-1)$
 - Time: $\log_2 N$



Today: Floating Point Representations



Recall: Positional Numeral Systems

- Combine symbols by multiplying each digit by the base to the power of the digit's position and adding
- Decimal:
 - $735_{10} = 7 \times 10^2 + 3 \times 10^1 + 5 \times 10^0$
- Binary:
 - $101_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$

Supporting Real Numbers

- Digits to the right of the point are multiplied by negative powers of the base

- Decimal:

- $735.48_{10} = 7 \times 10^2 + 3 \times 10^1 + 5 \times 10^0 + 4 \times 10^{-1} + 8 \times 10^{-2}$

- Binary:

- $101.101_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$

Recall: Decimal to Binary Conversion

- Conversion from decimal to binary can be done by successively dividing the number by two and extracting the remainder
- Example: converting 94_{10} to binary

$94 / 2 = 47$ remainder 0 (least significant digit)

$47 / 2 = 23$ remainder 1

$23 / 2 = 11$ remainder 1

$11 / 2 = 5$ remainder 1

$5 / 2 = 2$ remainder 1

$2 / 2 = 1$ remainder 0

$1 / 2 = 0$ remainder 1 (most significant digit)

Answer: 1011110_2

Converting Real Numbers

- The part to the right of the point can be converted by successively multiplying by two and extracting the digit to the left of the point
- Example: converting 0.78125_{10} to binary

$$0.78125 \times 2 = \underline{1}.5625$$

$$0.5625 \times 2 = \underline{1}.125$$

$$0.125 \times 2 = \underline{0}.25$$

$$0.25 \times 2 = \underline{0}.5$$

$$0.5 \times 2 = \underline{1}.0$$

Answer: 0.11001_2

Representing Real Numbers

- How to represent real numbers using a limited number of bits?
- One possibility: **fixed-point** representation
 - Dedicate a fixed set of bits for each of the values before and after the decimal
 - Example: In a 32-bit word, the first 16 bits represent the value before the decimal and the remaining 16 bits represent the value after the decimal
 - The *point* is at a *fixed* position in the N-bit word

Limitations of Fixed-Point

- Fixed-point representation has very limited range
 - If using 16 bits before and after the decimal:
 - Smallest number: $000\dots000.000\dots001 = 2^{-16} = 1.525878 \times 10^{-5}$
 - Largest number: $111\dots111.111\dots111 = 2^{16} - 2^{-16} = 6.553599998 \times 10^4$
- Cannot represent very small numbers
 - e.g., 6.67430×10^{-11} (gravitational constant)
- Cannot represent very large numbers
 - e.g., 6.0221409×10^{23} (Avogadro's number)

Normalized Numbers

- In decimal, **scientific notation** is used to represent numbers with wide range:
 - Examples:
 - 920,000,000 written as 9.2×10^8
 - 0.00000043 written as 4.3×10^{-7}
 - No need to write down all the zeros
 - Only need to represent the **significand** and **exponent**
- A number in scientific notation is **normalized** if it has exactly one non-zero digit before the point
 - Examples:
 - e.g., 9.2×10^8 is normalized
 - e.g., 92.0×10^7 and 0.92×10^9 are not normalized

Floating Point Representations

- The **floating point** representation for binary real numbers uses the concept of normalized scientific notation to represent numbers with a wide range
- Examples:
 - 1010100000000000 can be represented as 1.0101×2^{14}
 - 0.0000000001011 can be represented as 1.011×2^{-9}
- No need to store all the bits, just the significand and the exponent

Floating Point

- Representation for non-integral numbers
 - Including very small and very large numbers
- Like scientific notation
 - -2.34×10^{56} ← normalized
 - $+0.002 \times 10^{-4}$ ← not normalized
 - $+987.02 \times 10^9$ ← not normalized
- In binary
 - $\pm 1.xxxxxxx_2 \times 2^{yyyy}$
- Types `float` and `double` in C

Floating Point Standard

- Defined by IEEE Std 754-1985
- Developed in response to divergence of representations
 - Portability issues for scientific code
- Now almost universally adopted
- Two representations
 - **Single precision** (32-bit)
 - **Double precision** (64-bit)

IEEE Floating-Point Format

1 bit single: 8 bits
double: 11 bits

single: 23 bits
double: 52 bits

S	Exponent	Fraction
---	----------	----------

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- S: sign bit (0 $\Rightarrow$ non-negative, 1 $\Rightarrow$ negative)
- Normalize significand: $1.0 \leq |\text{significand}| < 2.0$
 - Always has a leading pre-binary-point 1 bit, so no need to represent it explicitly (hidden bit)
 - Significand is Fraction with the “1.” restored
- Exponent: excess representation: actual exponent + Bias
 - Ensures exponent is unsigned
 - Single: Bias = 127; Double: Bias = 1023

Floating-Point Example

- What number is represented by the single-precision float

11000000101000...00

- $S = 1$
- Fraction = $01000...00_2$
- Exponent = $10000001_2 = 129$
- $x = (-1)^1 \times (1.01_2) \times 2^{(129 - 127)}$
 $= (-1) \times 1.25 \times 2^2$
 $= -5.0$

Floating-Point Example

- Represent -0.75
 - $-0.75 = -0.11_2 = (-1)^1 \times 1.1_2 \times 2^{-1}$
 - $S = 1$
 - Fraction = $1000\dots00_2$
 - Exponent = $-1 + \text{Bias}$
 - Single: $-1 + 127 = 126 = 01111110_2$
 - Double: $-1 + 1023 = 1022 = 011111111110_2$
- Single: $10111111101000\dots00$
- Double: $101111111111101000\dots00$

Precision and Range

- **Precision** refers to the number of significant digits that can be represented
 - Impacted by the size of the fraction
- **Range** refers to the scope of numbers that can be represented (how large/small)
 - Impacted by the size of the exponent

Floating-Point Precision

- Single precision: 23 fraction bits
 - Equivalent to $23 \times \log_{10} 2 \approx 23 \times 0.3 \approx 6$ decimal digits of precision
- Double precision: 52 fraction bits
 - Equivalent to $52 \times \log_{10} 2 \approx 52 \times 0.3 \approx 16$ decimal digits of precision

Single-Precision Range

- Exponents 00000000 and 11111111 reserved
- Smallest value:
 - Smallest exponent: 00000001
 $\Rightarrow$ actual exponent = $1 - 127 = -126$
 - Smallest fraction: 000...00 $\Rightarrow$ significand = 1.0
 - Smallest value: $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- Largest value
 - Largest exponent: 11111110
 $\Rightarrow$ actual exponent = $254 - 127 = +127$
 - Largest fraction: 111...11 $\Rightarrow$ significand ≈ 2.0
 - Largest value: $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

Double-Precision Range

- Exponents 0000...00 and 1111...11 reserved
- Smallest value:
 - Smallest exponent: 000000000001
 $\Rightarrow$ actual exponent = $1 - 1023 = -1022$
 - Smallest fraction: 000...00 $\Rightarrow$ significand = 1.0
 - Smallest value: $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$
- Largest value:
 - Largest exponent: 111111111110
 $\Rightarrow$ actual exponent = $2046 - 1023 = +1023$
 - Largest fraction: 111...11 $\Rightarrow$ significand ≈ 2.0
 - Largest value: $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

Representing Zero

- The format discussed so far cannot represent 0.0 because of the implicit 1 to the left of the point in normalized numbers
 - Normalized numbers: $\pm 1.xxxxxxx_2 \times 2^{yyyy}$
- Solution: reserve the exponent 000...0 for **denormalized numbers** where the bit to the left of the point is implicitly 0
 - Denormalized numbers: $\pm 0.xxxxxxx_2 \times 2^{yyyy}$

Denormalized Numbers

- If Exponent = 000...0 $\Rightarrow$ hidden bit is 0

$$x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}+1}$$

- Zero can be represented as a denormalized number with fraction = 000...0

$$x = (-1)^S \times (0 + 0) \times 2^{-\text{Bias}+1} = \pm 0.0$$

Two representations
of 0.0!



Denormalized Numbers

- Denormalized numbers can also be used to represent numbers smaller than smallest normalized number
- Example:
 - Single precision, Fraction=00100...0
$$x = (-1)^S \times (0.00100...0) \times 2^{-127+1} = \pm 1.0 \times 2^{-129}$$
 - Recall: smallest normalized single precision number was: $\pm 1.0 \times 2^{-126}$
- Denormalized numbers allow for gradual underflow, with diminishing precision

Representing Unusual Events

- Need a way to represent special values that arise from unusual events, such as:
 - **Overflow** (the result of an operation is larger than the range the format can represent)
 - Represent as: **$\pm\text{infinity}$** ($\pm\infty$)
 - **Undefined result** (e.g., $0.0/0.0$, $\infty - \infty$)
 - Represent as: **NaN** (Not a Number)
- Solution: reserve the exponent $111\dots1$ for special values

Infinites and NaNs

- If Exponent = 111...1, Fraction = 000...0
 - $\pm$ Infinity
 - Can be used in subsequent calculations, avoiding need for overflow check
- If Exponent = 111...1, Fraction $\neq$ 000...0
 - Not-a-Number (NaN)
 - Indicates illegal or undefined result
 - Can be used in subsequent calculations

Textbook Sections

- The content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - 3.5